

From Vibe Coding to Agentic Engineering

Practical Vibe Coding, Shared Vocabulary, Context, Skills, Multi-Agent & Plugins (agy & claude)

What Is Inside This Concise Coursebook (5 Focused Sections & 6 Hands-On Labs)

- **Section 1 — Intro to Vibe Coding: Code as a Model of Understanding & Language as Action (Lab 01):** Why code is both machine instructions and a shared model of understanding (*What Is Code?*, martinfowler.com), Wittgenstein's **Two Theories of Language** (*Early Picture Theory* for formal code vs. *Late Meaning-as-Use* for using language to trigger actions, backed by Wang, Liang & Manning ACL 2016 arXiv:1606.02447 and Molino & Tagliabue arXiv:2302.01570), talking to AI like a coworker, Kief Morris's *Why Loop vs. How Loop*, and the 5-minute quickstart.
- **Section 2 — Persistent Project Context (CLAUDE.md / GEMINI.md) & Cross-Session Auto-Memory (Lab 02):** Fresh context windows, concise root context files (≤ 40 lines) with on-demand @docs/*.md references, git-worktree-shared Auto-Memory (~/.claude/projects/<project>/memory/MEMORY.md vs. Anti-gravity Knowledge Items at ~/.gemini/antigravity/knowledge/<project>/KNOWLEDGE.md with #direct tags), and /context (/stats) & /memory hygiene.
- **Section 3 — Crafting agentskills.io Skills Manually (“A Conciencia”) vs. Self-Generated Traps (Lab 03):** Skills as instruction manuals for AI, 3-Level Progressive Disclosure, the multi-file anthropic-brand/ skill (SKILL.md, docs.md, slides-deck.md, apply_template.md), and SkillsBench (arXiv:2602.12670) proof that manual skills lift pass rates by **+16.6 pp** while self-generated skills **degrade** pass rates by **−8.1 to −11.5 pp**.
- **Section 4 — Assessing Skills (eval-viewer) & Avoiding Skill Overload (Lab 04):** Paired with_skill vs. without_skill benchmarking (grading.json, benchmark.json, eval-viewer), solving the ≥ 4 skill overload penalty (**+19.0 pp** $\rightarrow$ **+10.1 pp**) by consolidating micro-skills into on-demand reference files, and calibrating trigger descriptions.
- **Section 5 — Multi-Agent Collaboration (feature-dev), Versioned Plugins + MCP & The SDD Bridge (Labs 05–06):** The 7-phase feature-dev workflow (code-explorer, Phase 3 Clarifying Questions gate, code-architect, and ≥ 80 -confidence code-reviewer), Skills vs. Plugins (plugin.json + MCP + PreToolUse hooks), and when to graduate to Spec-Driven Development (SDD-Crash-Course).

Contents

1	Intro to Vibe Coding: Code as a Model of Understanding & Language as Action	2
1.1	Wittgenstein's Two Theories of Language: Why We Use Language to Produce Actions	3
2	Persistent Project Context (CLAUDE.md / GEMINI.md) & Cross-Session Auto-Memory	6
3	Crafting agentskills.io Skills Manually (“A Conciencia”) vs. Self-Generated Traps	8
3.1	Empirical Proof from SkillsBench (arXiv:2602.12670): Why Self-Generated Skills Fail	8

4	Evaluating Skills (eval-viewer), Solving Skill Overload & Trigger Calibration	10
4.1	Paired Benchmarking (eval-viewer) & Signal Detection Theory (d' , c)	10
5	Multi-Agent Collaboration (feature-dev), Versioned Plugins + MCP & The SDD Bridge	12
5.1	Skills vs. Plugins & The “On-the-Loop” Bridge to Spec-Driven Development	12

1 Intro to Vibe Coding: Code as a Model of Understanding & Language as Action

Code Is Both Machine Instructions and a Model of Understanding (*What Is Code?*, [martinfowler.com](#), May 2026). In “*What Is Code?*” ([martinfowler.com/articles/what-is-code.html](#)), Unmesh Joshi articulates why software engineering discipline matters even more in the LLM era: as large language models make generating syntax cheaper, the mechanical act of typing instructions becomes less central. What becomes paramount is **making the conceptual model explicit, discovering the right domain vocabulary, and refining that vocabulary through iteration, domain expertise, and feedback**. Programming languages continue to matter deeply because we are not meant to be passive reviewers of generated diffs—the act of structuring code is itself part of our thinking.

Cognitive Debt: Why Passive Review of “Vibe-Coded” Output Fails. When developers treat an LLM as a black box (“Ralph Wiggum” vibe coding) and judge outputs solely by whether a quick demo runs, repositories rapidly accumulate **Cognitive Debt**:

- **Synonym & Vocabulary Drift:** Across three prompts, an ungrounded agent invents `client_id`, `cust_acct`, and `user_id` for the same entity, or mixes `tx_amt` and `fee_float` using IEEE-754 floating-point numbers instead of integer minor units (MoneyCents).
- **Plausible But Ungrounded Abstractions:** The LLM generates classes named `TransactionController` or `SettlementManager` that look syntactically idiomatic but violate double-entry conservation invariants.
- **Silent Guessing on Missing Context:** When an exchange rate or policy rule is absent from the prompt, a statistical next-token predictor silently guesses a default (such as `rate = 1.0`) rather than pausing to ask a clarifying question.

The Why Loop vs. Nested How Loops & “On-the-Loop” Harness Engineering

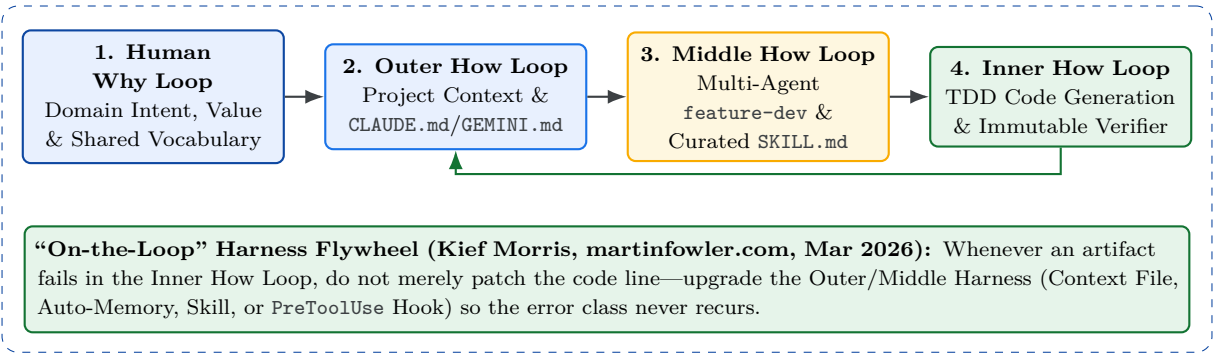


Figure 1: Separating the human-owned **Why Loop** from the agent-executed nested **How Loops** via On-the-Loop Harness Engineering.

repository (Vibe-Coding-Course) provides 1-to-1 branch parity across **Antigravity / Gemini CLI (agy branch)** and **Claude Code (claude branch)**, with 6 self-diagnosing Python labs that execute in under 1 second using zero external dependencies:

5-Minute Terminal Quickstart: Dual-Branch Setup (agy vs. claude) & Self-Diagnosis

```
# 1. Select your active coding agent harness branch
$ git checkout agy      # Antigravity / Gemini CLI (GEMINI.md, AGENTS.md, .agents/*)
$ git checkout claude   # Claude Code (CLAUDE.md, .claude/*, .claude-plugin/*)

# 2. Run the master self-diagnosis across all 6 labs (< 1 second, stdlib only)
$ ./self_diagnose_all.sh
# [ALL PASSED] 6/6 Hands-On Vibe-Coding-Course Labs Verified!

# 3. Launch learner mode in your chosen harness
$ agy --workspace .      # Antigravity Track
$ claude --settings .claude/learner.settings.json # Claude Code Track
```

1.1 Wittgenstein's Two Theories of Language: Why We Use Language to Produce Actions

Bridging Early Wittgenstein (Formal Code) and Late Wittgenstein (Language as Action). In Vibe Coding, we use natural language to trigger engineering actions and generate executable code. Ludwig Wittgenstein's two philosophies of language explain the exact tension every developer experiences when collaborating with an AI agent (Molino & Tagliabue, arXiv:2302.01570; Winograd & Flores, 1986):

1. **Theory 1 — Early Wittgenstein (*Tractatus Logico-Philosophicus*, 1921): The Picture Theory of Language.** Language is a strict, formal, 1-to-1 logical picture of facts. This is how **traditional programming languages, compilers, type systems, and unit tests** operate: an expression is either deterministically valid or rejected by the compiler.
2. **Theory 2 — Late Wittgenstein (*Philosophical Investigations*, 1953): Meaning as Use & Language Games.** Wittgenstein later realized that natural human language is not a static formal calculus, but a **toolbox used between collaborators to produce actions** (like the builder and assistant on a job site calling out “Block!”, “Pillar!”, “Slab!”). Words get their meaning from how they are used inside a shared team context (SciTePress 139777; MaKTO arXiv:2501.14225).
3. **Empirical Proof in Human–Computer Coding Games (Wang, Liang & Manning, ACL 2016, arXiv:1606.02447):** In “*Learning Language Games through Interaction*”, Stanford researchers modeled Wittgenstein's builder–assistant game across 100 humans who could only use natural language to instruct a computer agent that executed the actual actions (SHRDLURN). Two human strategies directly predicted task completion: **(a) Avoiding Synonyms** (using consistent domain vocabulary) and **(b) Compositionality** (building complex actions out of reusable defined instructions—the exact role of Skills). When context is missing, a well-designed agent pauses to ask coworker-style clarifying questions rather than guessing (Graziano, LGDL, 2025).

System Design Invariant & Tie-Breaker

Lab 01 Recipe: Shared Domain Vocabulary & Coworker Clarification (REQ-0101..0106)

- **Eliminate Synonym & Float Drift (REQ-0101..0102):** Replace untyped dicts (`client_id`, `tx_amt`) with frozen dataclasses `AccountId(r"~ACCT-[A-Z0-9]{4,12}$")` and `MoneyCents(amount_cents: int, currency: str)` that raise `TypeError` on float or bool.

Lab 01 Blueprint: Shared Domain Vocabulary & Clarification Gate (domain_ledger.py)

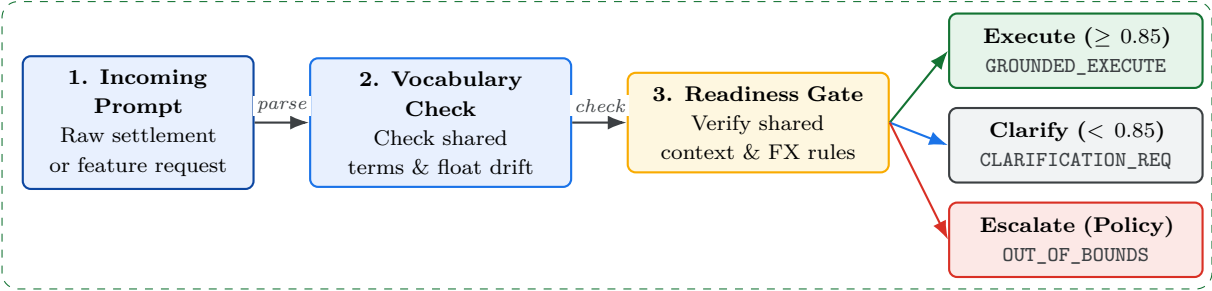


Figure 2: Bridging natural-language prompts (Late Wittgenstein) with strict domain types and coworker clarification checks (Early Wittgenstein) in Lab 01.

Foundational Reference	Core Analogy / Theory	Key Finding for Coding	Practical Vibe Coding Rule (Lab 01)
Joshi (2026) (<i>What Is Code?</i>)	Code as explicit Model of Understanding	Passive diff review causes Cognitive Debt & synonym drift	Encode explicit domain vocabulary in types (AccountId, MoneyCents).
Early vs. Late Wittgenstein (arXiv:2302.01570)	<i>Picture Theory</i> (formal logic) vs. <i>Meaning as Use</i> (toolbox)	LLMs use contextual language (<i>Investigations</i>) to emit strict code (<i>Tractatus</i>)	Talk to AI like a coworker: supply shared context (CLAUDE.md/GEMINI.md) so words map to valid code.
Wang, Liang & Manning (ACL 2016, arXiv:1606.02447)	Builder–Assistant language-to-action game (SHRDLURN)	Avoiding synonyms & compositionality drive task success across 100 players	Ban synonym drift (client_id vs. cust_acct) and package reusable actions as Skills .
Graziano (LGDL, 2025) & (MaKTO arXiv:2501.14225)	Action-grounded dialogue & Epistemic Honesty	Grounded utterances lift win rate +23% vs. GPT-4o ; ask when < 0.85	Never guess missing FX rates or policy limits; pause and return CLARIFICATION_REQUIRED.

Table 1: How code-as-understanding and Wittgenstein’s two theories of language translate into practical Lab 01 coding rules.

- **Enforce Double-Entry Conservation (REQ-0103):** `SettlementBatch.apply_postings()` verifies currency homogeneity, non-negative balances, and $\sum \text{debits} = \sum \text{credits}$.
- **Ask Clarifying Questions Instead of Guessing (REQ-0104..0105):** When a cross-currency request (JPY $\rightarrow$ USD) lacks a rate in `known_fx_basis_points`, `evaluate_request_readiness()` (aliased as `evaluate_language_game_move()`) returns `CLARIFICATION_REQUIRED` with coworker-style questions.

2 Persistent Project Context (CLAUDE.md / GEMINI.md) & Cross-Session Auto-Memory

Two Mechanisms Carry Knowledge Across Fresh Context Windows. Every Claude Code or Antigravity session begins with a completely **fresh context window**. Without persistent state, the agent suffers from session-to-session amnesia. Both harnesses solve this using two complementary file-backed mechanisms summarized in Table 2 and Figure 3:

1. **Human-Authored Persistent Project Context (CLAUDE.md on claude / GEMINI.md & AGENTS.md on agy):** Checked into Git to onboard every agent session with exact build/test commands, directory topology, architectural invariants, and coding conventions. Inside CLAUDE.md or GEMINI.md, engineers use **on-demand @path/to/file.md references** (e.g., @docs/architecture.md, @docs/testing-conventions.md) so deep documentation is pulled only when relevant—saving startup tokens just like progressive disclosure in skills.
2. **Agent-Authored Cross-Session Auto-Memory (MEMORY.md on claude / KNOWLEDGE.md on agy):** Notes the agent writes itself based on user corrections, debugging discoveries, and workflow preferences:
 - **Claude Code (claude branch):** Each project receives a dedicated memory directory at ~/.claude/projects/<project>/memory/. Crucially, the <project> path is derived from the **git repository root**, so **all git worktrees and subdirectories within the same repository share one unified auto-memory directory** (outside a git repo, the project root is used instead). Only the first **200 lines / 25 KB** of MEMORY.md are auto-loaded at session start; detailed notes are stored in separate topic files (topics/debugging.md) read on demand.
 - **Antigravity (agy branch):** The Antigravity Knowledge Base (~/.gemini/antigravity/knowledge/<project>/) stores workspace preferences in KNOWLEDGE.md (scope: workspace) and anchors every factual claim in topics/*.md with **epistemic provenance tags**: #direct (reserved *strictly* for explicit user instructions), #commit:<sha> (git commit anchor), #time:<YYYY-MM-DD>, and #session:<id>, inspected via /memory show and updated via /memory add or /memory refresh.

Lab 02 Architecture: Progressive Project Context (@path) & Git-Worktree Auto-Memory

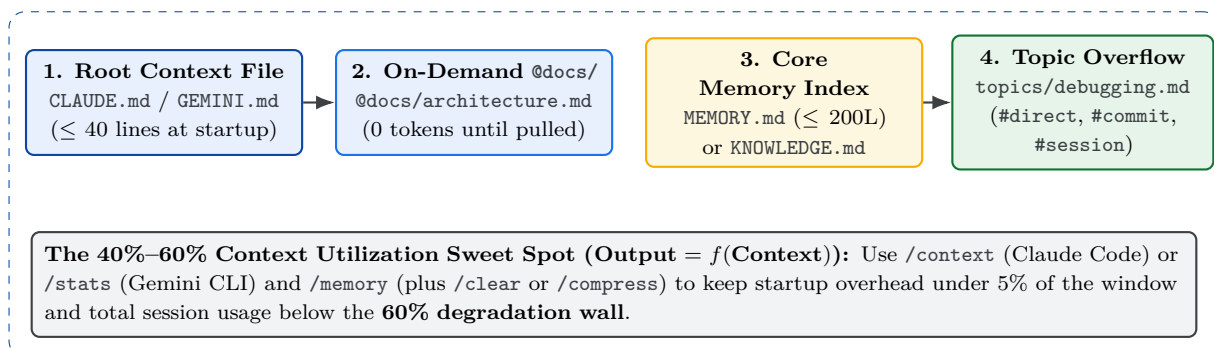


Figure 3: Separating concise root project context (CLAUDE.md / GEMINI.md with on-demand @docs/*.md imports) from cross-session Auto-Memory (MEMORY.md / KNOWLEDGE.md).

Copy-Pasteable Root Context & Epistemic Memory Discipline (Lab 02). In Lab 02 (context_memory_manager.py), you replace a 500-line monolithic context dump with a validated ≤ 40-line root context file and a git-worktree-aware memory compiler:

Dimension	Claude Code (claude Branch)	Antigravity / Gemini CLI (agy Branch)
Root Context File	CLAUDE.md (≤ 40 lines) + .claude/rules/*.md (paths: globs).	GEMINI.md & AGENTS.md (≤ 40 lines) + .agents/rules/*.md (always_on model_decision).
On-Demand Docs	@docs/architecture.md & @docs/ testing-conventions.md pulled when needed.	@docs/architecture.md mentions & progressive subdirectory AGENTS.md discovery.
Auto-Memory Path	~/.claude/projects/<repo-slug>/ memory/MEMORY.md (shared across git worktrees; first 200 lines / 25 KB auto-loaded).	~/.gemini/antigravity/knowledge/<project>/ KNOWLEDGE.md + topics/*.md Knowledge Items.
Provenance & CLI	/context (token audit), /memory (view/edit memory), /compact, /clear.	/stats (token audit), /memory (show add refresh), /skills, /compress, #direct tag discipline.

Table 2: 1-to-1 correspondence between Claude Code (claude) and Antigravity (agy) context and memory primitives.

Copy-Pasteable Template: Concise Root Context (CLAUDE.md / GEMINI.md) & Provenance Tags

```
# Payments Ledger Service - Persistent Project Context
## Commands & Environment
- Test runner: 'CI=true python3 -m unittest discover -v'
- Lab grader: './labs/<lab>/self_diagnose.sh [TARGET_DIR]'
## Architecture & Directory Map
- 'src/domain/': Ubiquitous Language value objects ('AccountId', 'MoneyCents').
## Core Conventions
- Monetary amounts MUST use integer minor units ('MoneyCents'), never floats.
## On-Demand References
- Pull architecture deep-dive on demand via @docs/architecture.md
- Pull testing & immutable verifier rules via @docs/testing-conventions.md

# Epistemic Provenance Discipline in Auto-Memory (REQ-0204):
- Always use MoneyCents integer arithmetic #direct #time:2026-10-08
- Batch worker uses SQLite WAL mode #commit:654321a #session:deadbeef (NO #direct tag!)
```

3 Crafting agentskills.io Skills Manually (“A Conciencia”) vs. Self-Generated Traps

Skills Are Procedural Instruction Manuals for AI. An agent skill is a modular, filesystem-based instruction manual (SKILL.md plus optional companion files, references/, scripts/, and assets/) that teaches an AI agent how to complete a specific class of tasks reliably. In both Claude Code (`.claude/skills/<name>/SKILL.md`) and Antigravity (`.agents/skills/<name>/SKILL.md`), you can invoke a skill manually via `/skillname` (for example, `/anthropic-brand` or `/pdf`) or allow the agent to load it automatically when your prompt matches its YAML frontmatter description.

The agentskills.io Specification & 3-Level Progressive Disclosure. To prevent a catalog of skills from exhausting the context window before work even begins, the open **Agent Skills Specification** (<https://agentskills.io/specification>) enforces a strict **3-Level Progressive Disclosure** architecture illustrated in Figure 4:

1. **Level 1 — Metadata** (≈ 100 tokens at session startup): Only the YAML frontmatter name (1..64 lowercase alphanumeric/hyphen characters matching `^[a-z0-9]+(-[a-z0-9]+)*$` and the parent directory name) and description (1..1024 characters specifying what the skill does, positive Use when... triggers, and negative Don't use for... boundaries) are injected into the startup system prompt.
2. **Level 2 — Core Instructions** (50..500 lines / $< 5,000$ tokens on activation): When invoked via `/skillname` or matched by the router, the agent reads the SKILL.md body containing core invariants and a routing table to companion files.
3. **Level 3 — On-Demand Resources** (0 startup tokens until needed): Companion reference manuals and scripts—such as docs.md, slides-deck.md, and apply_template.md inside the anthropic-brand/ directory—are loaded *only* for the specific deliverable being produced, saving $> 75\%$ of context tokens.

Lab 03 Anatomy: 3-Level Progressive Disclosure in anthropic-brand/

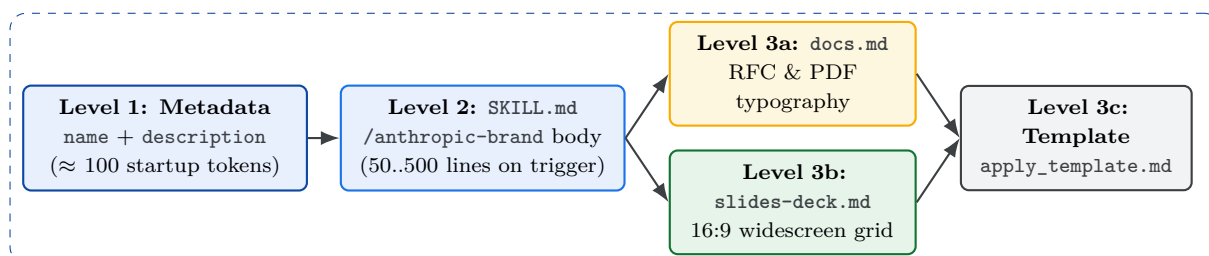


Figure 4: 3-Level Progressive Disclosure in the anthropic-brand/ skill directory: Level 1 metadata (≈ 100 tokens) $\rightarrow$ Level 2 SKILL.md router $\rightarrow$ Level 3 artifact-specific reference files.

3.1 Empirical Proof from SkillsBench (arXiv:2602.12670): Why Self-Generated Skills Fail

Human-Crafted Skills (“A Conciencia”) vs. Self-Generated Skills. In *SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks* (arXiv:2602.12670v4, Li et al., Jun 2026), researchers evaluated **87 tasks across 8 domains** and **9,396 agent trajectories** across Claude Code, Gemini CLI, and Codex CLI. Their findings establish the central empirical law of skill engineering summarized in Table 3:

- **Curated Human-Authored Skills Boost Pass Rates by +16.6 pp:** Carefully authored skills raise average pass rates from **33.9% to 50.5%** (+25.5% normalized gain)—including **+24.8 pp on Gemini CLI + Gemini 3.1 Pro** (36.0% $\rightarrow$ 60.8%), **+18.2 pp on Claude Code + Opus 4.7** (43.0% $\rightarrow$

61.2%), and up to **+67.0 pp on procedural tasks** (such as `llm-prefix-cache-replay`, which jumps from 1.9% to 94.4%, a **+92.6 pp** lift).

- **Self-Generated Skills DECREASE Performance Below Baseline:** When an agent is instructed to generate its own skill before solving a task, pass rates drop **below the No-Skills baseline across every harness**: **−8.1 pp on Claude Code + Opus 4.7** (43.0% → 34.9%), **−11.3 pp on Codex + GPT-5.5** (46.8% → 35.5%), and **−11.5 pp on Gemini CLI + Gemini 3.1 Pro** (36.0% → 24.5%).
- **Why Self-Generated Skills Hurt:** An LLM generating a skill zero-shot can only regurgitate its own pretraining priors (“*write clean, modular code*”) or lock in **confidently wrong hallucinated gotchas**—such as SkillsBench’s `3d-scan-calc` failure, where a self-generated skill fabricated a rule to multiply all mesh dimensions by `1000.0` (assuming meters-to-millimeters on unitless coordinates), causing the downstream solver to fail deterministically.

Model–Harness Pair (SkillsBench)	No Skills	Curated Skills	Self-Generated	Empirical Takeaway
Claude Code + Opus 4.7	43.0%	61.2% (+18.2 pp)	34.9% (−8.1 pp)	Human-written skills add +18.2 pp; self-gen hurts by −8.1 pp.
Gemini CLI + Gemini 3.1 Pro	36.0%	60.8% (+24.8 pp)	24.5% (−11.5 pp)	Largest curated lift (+24.8 pp); self-gen drops −11.5 pp.
Codex CLI + GPT-5.5	46.8%	62.9% (+16.1 pp)	35.5% (−11.3 pp)	Consistent degradation when LLM writes its own skill.

Table 3: Empirical pass rates from SkillsBench (arXiv:2602.12670v4, Table 6 & Fig. 10) proving why skills must be authored manually (“*a conciencia*”) from real failure trajectories.

Common Architectural Antipatterns & Diagnostic Traps

Experience Before Theory: How to Write High-Signal Skills (Lab 03 REQ-0301..0306) **Never ask an LLM to generate a skill from scratch out of thin air.** Instead, follow the 3-step **Experience Before Theory** protocol tested in Lab 03 (`skill_validator.py`):

1. **Run Zero-Shot First:** Execute representative prompts *without* any skill and record exact failure trajectories (e.g., failing WCAG contrast by pairing `#D97757` on `#5E5D59`).
2. **Encode Only the Minimal Prescriptive Delta:** Write concrete commands, exact hex tokens (`#FAF9F5`, `#141413`), and causal *why* explanations—stripping generic training-data filler.
3. **Split Reference Files One Level Deep:** Keep `SKILL.md` between 50 and 500 lines and place artifact-specific rules in `docs.md`, `slides-deck.md`, and `apply_template.md`.

4 Evaluating Skills (eval-viewer), Solving Skill Overload & Trigger Calibration

Why Adding More Skills Confuses the Agent (SkillsBench Table 8 & Table 9). A common failure mode among teams adopting Claude Code or Antigravity is creating dozens of tiny, overlapping skills (git-commit-msg, git-branch-create, git-rebase-helper, git-pr-upload). Because every skill's Level 1 name and description are injected into the startup system prompt, adding more skills degrades routing precision:

- **The Skill Count Sweet Spot (2-3 Skills):** In SkillsBench Table 8, providing 1 skill yields +18.0 pp lift and 2-3 skills yields the peak +19.0 pp lift, whereas providing ≥ 4 skills drops lift to +10.1 pp (an 8.9 percentage-point collapse due to trigger confusion and conflicting instructions).
- **The Complexity Sweet Spot (Compact & Standard Beat Exhaustive):** In SkillsBench Table 9, Compact (+19.0 pp) and Standard (+21.5 pp) skills dramatically outperform Comprehensive/Exhaustive manuals (+0.7 pp), which bury critical gotchas in thousands of tokens of noise.
- **The Reference-First Consolidation Rule:** Never create a standalone micro-skill under 50 lines. Consolidate sibling sub-workflows into references/<subtopic>.md inside a single Complete User Outcome (CUJ) parent skill—adding 0 startup tokens and 0 new routing targets.

Left-to-Right Decision Tree: Skip vs. Reference File vs. Standalone Skill vs. Versioned Plugin

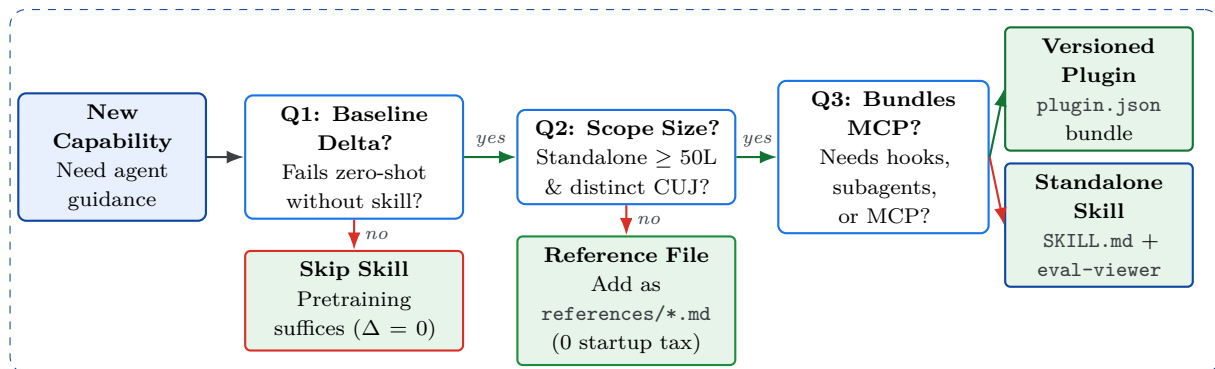


Figure 5: Left-to-Right architectural decision tree preventing skill catalog overload and deciding between references/*.md, a standalone SKILL.md, and a versioned plugin.json bundle.

4.1 Paired Benchmarking (eval-viewer) & Signal Detection Theory (d' , c)

Anthropic's skill-creator & eval-viewer Workflow. To verify empirically whether a skill improves agent performance, Anthropic's skill-creator (skills/skill-creator/SKILL.md) and eval-viewer execute a 4-step paired evaluation loop implemented in Lab 04 (skill_eval_engine.py):

1. **Paired Parallel Runs:** Spawn with_skill and without_skill trajectories on the exact same test prompts.
2. **Assertion Grading (grading.json):** Grade each expectation into {"text": str, "passed": bool, "evidence": str}.
3. **Benchmark Aggregation (benchmark.json) & The 11/11 Zero-Flake Rule:** Compute mean/stddev pass rates, token deltas, and net lift. By binomial reliability ($p^n \leq 1 - C$ for target pass rate $p = 0.90$), 3/3 passes gives only 34% confidence that true reliability is $\geq 90\%$, whereas 11/11 **consecutive passes** achieves 72% statistical confidence.

4. **Visual Review** (`eval-viewer/generate_review.py`): Render a self-contained static HTML viewer with an **Outputs tab** (for inspecting qualitative transcripts and leaving `feedback.json` notes) and a **Benchmark tab** (for quantitative pass-rate and token-cost comparison).

Mathematical Trigger Calibration via Signal Detection Theory (d' and Criterion Bias c).

Optimizing a skill's description using raw Hit Rate alone falls into the **Trade-Off Trap**: making a description overly broad raises Hit Rate from 85% $\rightarrow$ 100% while False Alarms jump from 5% $\rightarrow$ 40% (polluting context on unrelated prompts). Using Hautus (1995) +0.5 smoothing on a 60/40 train/test split of positive and near-miss negative queries:

$$\begin{aligned} \text{HR}_{\text{adj}} &= \frac{H + 0.5}{H + M + 1.0}, & \text{FA}_{\text{adj}} &= \frac{FA + 0.5}{FA + CR + 1.0}, \\ d' &= \Phi^{-1}(\text{HR}_{\text{adj}}) - \Phi^{-1}(\text{FA}_{\text{adj}}), & c &= -\frac{1}{2} (\Phi^{-1}(\text{HR}_{\text{adj}}) + \Phi^{-1}(\text{FA}_{\text{adj}})) \end{aligned} \quad (1)$$

where **Sensitivity** d' measures true separation between positive intent and distractors ($d' > 2.0$ indicates strong discriminability), **Criterion Bias** c measures threshold conservatism ($c > +0.25$ = under-triggering; $c < -0.25$ = liberal false alarms; $c \approx 0$ = balanced), and **Utility** $U = 2.0d' - 0.5|c - c_{\text{target}}|$ selects the Pareto-optimal description.

Evaluation Dimension	Optimal Regime	Failure Regime	Engineering Rule (<code>skill_eval_engine.py</code>)
Active Skill Count (SkillsBench Table 8)	2-3 Skills (+ 19.0 pp) or 1 Skill (+ 18.0 pp)	≥ 4 Skills drops lift to + 10.1 pp (−8.9 pp)	Cap active skills per workspace at 2-3; merge micro-skills into <code>references/*.md</code> .
Skill Length / Detail (SkillsBench Table 9)	Standard (+ 21.5 pp) or Compact (+ 19.0 pp)	Comprehensive drops lift to + 0.7 pp	Keep <code>SKILL.md</code> under 500 lines; record only the Delta from baseline.
Zero-Flake Gate (Binomial $p^n \leq 1 - C$)	11/11 passes (72% conf. at $p = 0.90$)	3/3 passes (34% conf. only)	Require positive net pass-rate lift with zero regressions on baseline passes.
Trigger Calibration (SDT d' & Bias c)	$d' > 2.0$, $ c \leq 0.25$ (Balanced)	$c > +0.25$ (conservative) or $c < -0.25$ (noisy)	Optimize description via $U = 2.0d' - 0.5 c $ on a 60/40 train/test split.

Table 4: Quantitative skill catalog hygiene and trigger calibration thresholds implemented in Lab 04 (`skill_eval_engine.py`).

5 Multi-Agent Collaboration (feature-dev), Versioned Plugins + MCP & The SDD Bridge

Why Multi-Agent Collaboration Beats Single-Window Vibe Coding. When a single agent explores a repository, compares three competing architectural designs, implements code, and reviews its own diff in one conversation, raw grep outputs and abandoned branches push the context window past the **60% degradation threshold** and induce confirmation bias. Both Claude Code (via the official feature-dev plugin) and Antigravity (via `.agents/workflows/feature-dev.md`, `.agents/agents/*.md`, and `/plan`) solve this by delegating exploration, architecture, and review to **context-isolated read-only subagents** across the **7-Phase feature-dev Pipeline** illustrated in Figure 6:

1. **Phase 1 — Discovery:** Clarify the feature request and initialize a tracked 7-phase checklist.
2. **Phase 2 — Parallel Codebase Exploration (code-explorer subagents):** Spawn 2–3 parallel read-only code-explorer agents (tools: `Glob`, `Grep`, `Read`) that trace entry points and call chains using exact `file:line` citations and return a deduplicated list of essential files to read.
3. **Phase 3 — Clarifying Questions Gate (MANDATORY STOP):** Synthesize all underspecified edge cases, error contracts, and compatibility questions discovered in Phase 2 and **halt execution** (`ClarificationGateError`) until the human answers every question.
4. **Phase 4 — Tri-Lens Architecture Design (code-architect subagents):** Spawn parallel code-architect agents proposing three explicit blueprints: `minimal_changes` (smallest diff, maximum reuse), `clean_architecture` (strongest abstractions & testability), and `pragmatic_balance` (speed + quality). Wait for explicit human selection before touching code.
5. **Phase 5 — Implementation:** Implement the human-approved architecture and run `./labs/<lab>/self_diagnose.sh`.
6. **Phase 6 — Confidence-Filtered Quality Review (code-reviewer subagents):** Spawn 3 parallel code-reviewer agents auditing *Simplicity/DRY*, *Functional Bugs*, and *CLAUDE.md/GEMINI.md Conventions*, suppressing noise via a strict `confidence >= 80` filter (0..100 scale).
7. **Phase 7 — Summary:** Document files modified, architectural decisions, and deterministic test results.

Labs 05–06 Blueprint: 7-Phase feature-dev Multi-Agent Pipeline & Versioned Plugin Bundle

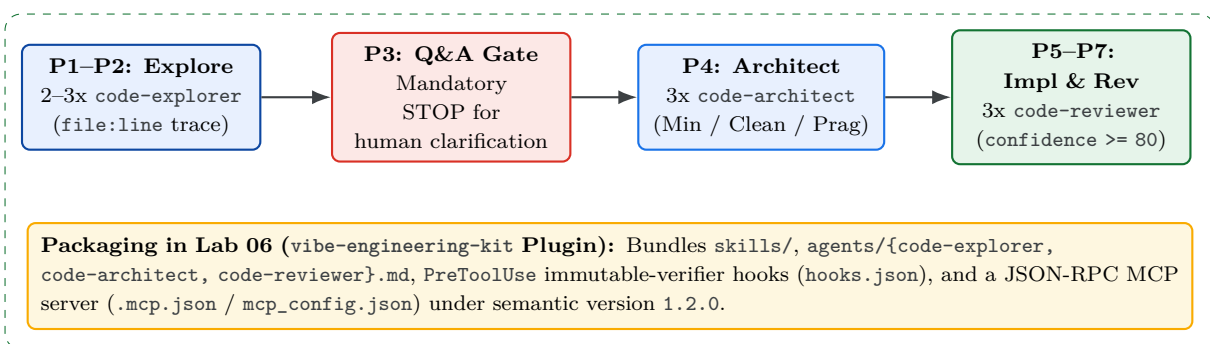


Figure 6: The 7-phase feature-dev multi-agent orchestration pipeline (Lab 05) packaged inside a versioned plugin with `PreToolUse` hooks and an MCP tool server (Lab 06).

5.1 Skills vs. Plugins & The “On-the-Loop” Bridge to Spec-Driven Development

Skills vs. Plugins: When to Bundle. While a **Skill** (`SKILL.md`) is a single Markdown instruction manual, a **Plugin** (`.claude-plugin/plugin.json` on claude / `.agents/plugins/<name>/plugin.json` on agy) is

a **versioned, namespaced distribution package** (/plugin-name:skill-name) that bundles skills/, custom agents/, commands/ or rules/, deterministic PreToolUse/Stop lifecycle hooks (hooks.json), and **MCP servers** (.mcp.json / mcp_config.json).

Observed Agent Failure	Flywheel Target Layer	Target Artifact (claude / agy)	Corrective Engineering Action
Wrong build/test command	ROOT_CONTEXT_RULE	CLAUDE.md / GEMINI.md	Add exact one-line test command to top of root context file.
Repeated preference drift	AUTO_MEMORY	MEMORY.md / KNOWLEDGE.md	Record explicit user correction with #direct provenance tag.
Procedural workflow error	CURATED_SKILL	SKILL.md + references/*.md	Encode observed failure delta & verify lift in eval-viewer.
Edited locked test suite	PRE_TOOL_USE_HOOK	hooks.json (PreToolUse)	Fail closed (decision: "block") on adversarial_tests/.
Missing live schema lookup	PLUGIN_MCP_TOOL	plugin.json + .mcp.json	Expose deterministic JSON-RPC tool via bundled MCP server.
≥ 3 -Service contract drift	ESCALATE_TO_SDD_SPEC	openspec/specs/ + conductor/	Transition from conversational Vibe Coding to Spec-Driven Development (SDD-Crash-Course).

Table 5: The On-the-Loop Harness Flywheel routing matrix (Lab 06 harness_flywheel.py) and the exact escalation boundary to Spec-Driven Development.

System Design Invariant & Tie-Breaker

60-Second Executive Cheat Sheet: 5 Rules to Master Before Starting the SDD Course

1. **Talk to AI Like a Coworker & Share Domain Vocabulary:** Bridge Late Wittgenstein (*language as action*) and Early Wittgenstein (*formal code*) by encoding domain terms into types (AccountId, MoneyCents) and requiring clarifying questions (CLARIFICATION_REQUIRED) instead of guessing (**Lab 01**).
2. **Keep Root Context ≤ 40 Lines with @path Imports:** Delegate deep docs via @docs/*.md and share Auto-Memory (≤ 200 lines) across git worktrees with strict #direct provenance (**Lab 02**).
3. **Write Skills Manually (“A Conciencia”), Never Self-Generated:** Curated skills lift pass rates by **+16.6 pp**; self-generated skills drop pass rates by **−8.1 to −11.5 pp** (**Lab 03**).
4. **Cap Active Skills at 2-3 & Verify with eval-viewer:** Consolidate micro-skills into references/*.md to avoid the ≥ 4 skill overload drop (**+19.0 pp $\rightarrow$ +10.1 pp**) (**Lab 04**).
5. **Orchestrate Subagents (feature-dev), Bundle Plugins & Spin the Harness Flywheel:** Enforce the Phase 3 Q&A gate and Phase 6 ≥ 80 review filter, package MCP + hooks into plugin.json, and escalate ≥ 3 -service brownfield changes to **Spec-Driven Development** (**Labs 05–06**).